Tecnología para la Gestión II

Clase 7 — Transformar variables: `mutate()` y `case_when()`

Francisco Villarroel Riquelme | 2026-10-07

Administración Pública · Escuela de Gobierno · Facultad de Economía y Gobierno · ADPU B012

¿Qué haremos hoy?

Bloque 1 — Transformar variables

- `mutate()`: crear y modificar columnas
- `case_when()`: recodificar según condiciones
- Manos a la obra con la base municipal

Bloque 2 — Estadística descriptiva

Ya sabemos elegir... ahora, crear

`select()`

Elige **columnas**.

`filter()`

Elige **filas**.

`mutate()`

Crea o modifica columnas.

Hasta ahora reducíamos la base. Con `mutate()` empezamos a **construir información nueva** a partir de la que ya tenemos: montos por persona, tramos, indicadores, etiquetas...

`mutate()` — crear una columna

Genera una variable nueva a partir de las que ya existen:

```
datos <- datos %>%
  mutate(monto_millones = monto_asignado / 1000000)
```

Se lee: *"a `datos`, agrégale una columna llamada `monto_millones` que sea `monto_asignado` dividido por un millón"*. La base conserva todas sus columnas y suma la nueva al final.

Recuerda **reasignar** con `datos <- datos %>% ...` para que el cambio quede guardado en la base.

`mutate()` — varias columnas a la vez

```
datos <- datos %>%
  mutate(costo_beneficiario = monto_asignado / n_beneficiarios,
         es_alta            = nivel_prioridad == "Alta")
```

`costo_beneficiario`

Una variable **numérica**: cuánto se gasta por persona beneficiada.

`es_alta`

Una variable **lógica** (`TRUE`/`FALSE`): ¿la prioridad es Alta? Nace de una condición.

Dentro de `mutate()` puedes crear tantas columnas como quieras, separadas por comas.

Recodificar con `case_when()`

El problema que resuelve `case_when()`

> La variable `estado` viene como número: 1, 2, 3, 4. Para un informe, nadie entiende "4": queremos que diga **"Rechazada"**.

`case_when()` traduce **condiciones en valores**. Es como decir:

> *"**Si** el estado es 1, ponle 'Ingresada'; **si** es 2, ponle 'En proceso'; **si** es 3, 'Resuelta'; **si** es 4, 'Rechazada'."*

`case_when()` — la sintaxis

```
datos <- datos %>%
  mutate(estado_txt = case_when(
    estado == 1 ~ "Ingresada",
    estado == 2 ~ "En proceso",
    estado == 3 ~ "Resuelta",
    estado == 4 ~ "Rechazada"))
```

Cada línea es **`condición ~ valor`**: a la izquierda del `~` va la pregunta (lógica), a la derecha el valor que asigna si se cumple. Se evalúan **de arriba hacia abajo** y gana la **primera** que se cumple.

El símbolo `~` se llama "virgulilla" o *tilde*. `case_when()` vive dentro de `mutate()`, porque estamos creando una columna.

`case_when()` — tramos o categorías

Sirve para agrupar una variable numérica en categorías:

```
datos <- datos %>%
  mutate(tramo_monto = case_when(
    monto_asignado <  1000000  ~ "Bajo",
    monto_asignado < 10000000  ~ "Medio",
    TRUE                       ~ "Alto"))
```

La última línea, **`TRUE ~ "Alto"`**, es el "para todo lo demás": atrapa cualquier caso que no cumplió las condiciones anteriores. **Sin ese `TRUE`, esos casos quedan como `NA`.**

Como se evalúa en orden, un monto de 500.000 ya calza en "Bajo" y no sigue bajando.

`case_when()` — condiciones combinadas

Puedes usar los operadores lógicos que ya conoces (`&`, `|`, `!=`):

```
datos <- datos %>%
  mutate(gestion = case_when(
    nivel_prioridad == "Alta" & estado != 4 ~ "Urgente",
    nivel_prioridad == "Media"              ~ "Seguimiento",
    TRUE                                    ~ "Puede esperar"))
```

> Traducción: *"si es prioridad Alta **y** no está rechazada → 'Urgente'; si es Media → 'Seguimiento'; en cualquier otro caso → 'Puede esperar'."*

Todo junto: `mutate` + `case_when` + pipe

```
datos <- datos %>%
  mutate(
    monto_millones = monto_asignado / 1000000,
    estado_txt = case_when(
      estado == 1 ~ "Ingresada",
      estado == 2 ~ "En proceso",
      estado == 3 ~ "Resuelta",
      estado == 4 ~ "Rechazada"),
    tramo_monto = case_when(
      monto_asignado <  1000000 ~ "Bajo",
      monto_asignado < 10000000 ~ "Medio",
      TRUE                      ~ "Alto"))
```

Un solo `mutate()` puede crear columnas simples y `case_when()` a la vez. Ordenado y legible.

Errores frecuentes con `case_when()`

De sintaxis

- Usar `=` o `==` en vez de `~` para asignar
- Escribir `=` en la condición (va `==`)
- Olvidar cerrar un paréntesis

De lógica

- Olvidar el `TRUE ~ ...` → aparecen `NA`
- Mezclar tipos en las salidas (unas texto, otras número)
- No envolverlo en `mutate()`

Todas las salidas de un mismo `case_when()` deben ser **del mismo tipo** (todas texto, o todas número).

Manos a la obra

Ejercicios con la base municipal

Ejercicios

1. Con `mutate()`, crea `semanas_tramitacion = dias_tramitacion / 7`.

2. Con `mutate()`, crea la variable lógica `urgente = nivel_prioridad == "Alta" & requiere_visita == "Sí"`.

3. Con `case_when()`, crea `estado_txt` con las etiquetas del estado (1 a 4).

4. Con `case_when()`, crea `tramo_beneficiarios`: menos de 100 → "Pocos"; menos de 1.000 → "Medio"; el resto → "Muchos".

Tarea: el código con 10 errores nivel experto

Ahora que ya conoces `mutate()` y `case_when()`, un desafío para la casa. Este script los combina con `select()` y `filter()`. Tiene **10 errores** —de sintaxis y de lógica—. Encuéntralos y corrígelo tú mismo.

```
install.packages(tidyverse)
Library(tidyverse)

datos <- read.csv(data/gestion_municipal.csv)

datos %>%
  select(comuna, tipo_solicitud, monto_asignado, estado, nivel_prioridad)
  filter(estado = 3 & monto_asignado > 500000) %>%
  mutate(monto_millones = monto_asignado / 1000000
        estado_txt = case_when(
          estado == 1 ~ "Ingresada"
          estado == 2 ~ "En proceso",
          estado == 3 ~ "Resuelta",
          estado == 4 ~ "Rechazada"),
        tramo = case_when(
          monto_asignado < 1000000  ~ "Bajo",
          monto_asignado < 10000000 ~ "Medio")) %>%
```

La próxima clase: Estadística descriptiva

Dudas: fvillarroelr1@docente.uss.cl · Tecnología para la Gestión II · ADPU B012

Pauta de ejercicios solo docente

```r
# 1) mutate simple
datos <- datos %>%
  mutate(semanas_tramitacion = dias_tramitacion / 7)

# 2) mutate lógico
datos <- datos %>%
  mutate(urgente = nivel_prioridad == "Alta" & requiere_visita == "Sí")

# 3) case_when: etiquetas
datos <- datos %>%
  mutate(estado_txt = case_when(
    estado == 1 ~ "Ingresada",
    estado == 2 ~ "En proceso",
    estado == 3 ~ "Resuelta",
    estado == 4 ~ "Rechazada"))

# 4) case_when: tramos
datos <- datos %>%
  mutate(tramo_beneficiarios = case_when(
    n_beneficiarios <  100 ~ "Pocos",
    n_beneficiarios < 1000 ~ "Medio",
    TRUE                   ~ "Muchos"))
```

Pauta: tarea de 10 errores solo docente

Sintaxis (6)

1. `"tidyverse"` sin comillas
2. `Library` → `library`
3. ruta del csv sin comillas
4. falta `%>%` después de `select()`
5. `estado = 3` → `estado == 3`
6. falta coma antes de `estado_txt =`

Sintaxis y lógica (4)

1. falta coma dentro del primer `case_when()`
2. `tramo` sin `TRUE ~ ...` final → NA en montos ≥ 10M
3. `%in% "Maipú", "Renca"` → falta `c(...)`
4. el resultado nunca se reasigna a `datos` (falta `datos <-`)

```
datos <- datos %>%
  select(comuna, tipo_solicitud, monto_asignado, estado, nivel_prioridad) %>%
  filter(estado == 3 & monto_asignado > 500000) %>%
  mutate(monto_millones = monto_asignado / 1000000,
         estado_txt = case_when(
           estado == 1 ~ "Ingresada",
           estado == 2 ~ "En proceso",
           estado == 3 ~ "Resuelta",
           estado == 4 ~ "Rechazada"),
         tramo = case_when(
```